

Interacció i modificació d'estat

Jordi Forga

Tardor 2025

Índex

1	Interacció i modificació d'estat	2
1.1	Creació d'una aplicació interactiva	2
1.2	Realització del joc	4
1.2.1	Primer pas	4
1.3	Segon pas	6
1.4	Tercer pas	7
1.5	Quart pas	9
1.6	Cinquè pas	10
2	Codi font del projecte	11
2.1	Configuració del projecte <i>stack</i>	11
2.2	Paquet life	11
2.2.1	Descripció del paquet	11
2.2.2	Mòdul Main	12
2.2.3	Mòdul Life.Board	13
2.2.4	Mòdul Life.Draw	15

1 Interacció i modificació d'estat

En aquesta pràctica realitzarem una simulació del [Joc de la Vida](#), inventat per John Conway l'any 1970. Consistirà en una aplicació que visualitza gràficament l'estat del joc (conjunt de cel·les vives) i en la que l'usuari podrà interactuar modificant l'estat, visualitzant-se el nou estat resultant.

IMPORTANT: Per realitzar aquesta pràctica cal que us situeu en el directori del projecte `prac2-prj`.

1.1 Creació d'una aplicació interactiva

Però ens podem preguntar: Cóm podem tenir interacció en un món sense efectes laterals ? Cóm podem guardar l'estat actual de l'aplicació en un món sense variables modificables ?

Un programa interactiu canvia l'estat quan es produeix un nou esdeveniment d'entrada. Bàsicament doncs, l'aplicació la podrem veure com una funció pura que obté el nou estat a partir de l'esdeveniment i de l'estat actual.

La biblioteca *Drawing* usada en aquesta pràctica ofereix la funció `activityOf` amb el següent tipus:

```
activityOf :: Int
           -> Double -> Double
           -> model
           -> (action -> State model ())
           -> (model -> View action
           -> IO ())
```

Aquesta és una declaració polimòrfica, on `model` és la variable de tipus corresponent a l'estat o model de l'aplicació, i `action` és la variable de tipus corresponent a les accions de l'aplicació.

La funció té els 6 paràmetres següents:

- El port HTTP al que s'haurà de connectar el vostre navegador.
- Les dimensions (amplada i alçada) en unitats gràfiques que tindrà l'àrea de visualització. Cal tenir en compte que cada unitat ocupa 20 pixels i que l'origen de coordenades està en el centre de l'àrea. Valor adequats d'aquests paràmetres per a la majoria de navegadors són 60.0 i 30.0 respectivament.
- L'estat inicial del model de l'aplicació.
- La funció que a partir de l'acció obté el monad que modifica el model de l'aplicació.
- Y la funció que obté la vista a partir de l'estat actual. La vista consisteix en:
 - la sortida gràfica (de tipus [Drawing](#)).

- les propietats de la vista, bàsicament la configuració de les funcions que obtenen les accions a partir dels esdeveniments d'interès (ratolí, teclat, etc.).

La funció `activityOf` obté una aplicació (servei HTTP) que mantindrà l'estat i processarà els esdeveniments generats des d'una pàgina HTML que interacciona amb l'usuari.

Veiem un exemple molt simple d'aplicació interactiva. Dins el directori del paquet `life`, teniu el fitxer `exemple.hs` amb el següent contingut:

```
module Main where
import Drawing
import Drawing.Activity

main :: IO ()
main = activityOf 3708 60 30 initial handle view

initial = 0

handle :: () -> State Double ()
handle () =
    modify $ \angle -> angle + pi / 4

view :: Double -> View ()
view angle =
    drawingView [ onKeyDown (const ()) ] draw
    where
        draw = rotated angle $
            polyline [(0, 0), (2, 0)]
            <> translated 2 0 (colored red $ solidCircle 0.5)
```

En aquest exemple observem que:

- L'estat de l'aplicació és un número de tipus `Double` que representa un angle en radians.
- L'estat inicial és 0 radians.
- La única acció d'aquest exemple, de tipus `()`, es produeix amb l'esdeveniment de tecla pulsada. Amb aquesta acció s'incrementa l'angle en `pi/4` radians.
- La visualització de l'estat (angle) consisteix en un simple dibuix rotat.

L'aplicació s'executarà fent:

```
stack runghc exemple.hs
```

L'aplicació consisteix en un servidor HTTP pel port 3708.

Un cop està executant, amb un navegador¹ visiteu l'exemple amb la URL `http://localhost:3708/`

¹Hi ha problemes amb el navegador `firefox`. Obriu la URL amb el navegador `chromium`.

1.2 Realització del joc

La pràctica es realitzarà en 5 passos. En cadascun dels passos anirem introduïnt noves funcionalitats fins arribar a la versió final (podeu provar el [funcionament de la versió final](#)).

El joc consisteix en una aplicació formada pel mòdul principal `Main` (fitxer `life.hs`) i els mòduls `Life.Board` i `Life.Draw` (fitxers `Life/Board.hs` i `Life/Draw.hs`). Els mòduls `Life.Board` i `Life.Draw` contenen la definició del taulell del joc i de la funció que el dibuixa respectivament.

La definició del taulell del joc i de les seves funcions es dona ja feta en el mòdul `Life.Board` (veieu el fitxer [life/Life/Board.hs](#) del projecte). Aquest mòdul defineix el tipus `Board` i les funcions següents:

```
— The type of the board
data Board = ...

— Each cell in the board is determined by a pair of Int's (column,
row)
type Pos = (Int, Int)

— The empty board (all cells are dead)
initBoard :: Board

— 'cellIsLive pos board' is true iff the cell at 'pos' of 'board'
is live
cellIsLive :: Pos -> Board -> Bool

— 'liveCells board' is the list of the positions of the live cells
of 'board'
liveCells :: Board -> [Pos]

— Get the minimum position (column, row) of the live cells
minLiveCell :: Board -> Pos

— Get the maximum position (column, row) of the live cells
maxLiveCell :: Board -> Pos

— 'setCell live pos board' change the liveness of cell at 'pos' of
'board' (to live if 'live' is 'True' or to dead if 'live' is '
False')
setCell :: Bool -> Pos -> Board -> Board

— 'nextGeneration board' gets the next generation of 'board'
nextGeneration :: Board -> Board
```

1.2.1 Primer pas

En el primer pas de la pràctica l'estat del joc consistirà senzillament en el taulell del joc.

El mòdul principal d'aquest primer pas (veieu el fitxer [life/life.hs](#) del projecte, i que ja se us dona fet) defineix el tipus `Game` per a l'estat del joc, el

tipus `Action` per a les accions del joc i conté el punt d'inici de l'aplicació i les funcions d'inicialització, tractament de les accions i visualització de l'estat.

S'ha definit el tipus `Game` corresponent a l'estat del joc (usant *sintaxis record*) amb un sol camp corresponent al taulell del joc:

```
data Game = Game
    { gmBoard :: Board      — last board generation
    }
```

La funció:

```
gmBoard :: Game -> Board      — Aquesta funcio es deriva de
                                — la sintaxis record
```

ens permetrà obtenir el taulell de l'estat del joc.

Les úniques accions que tindrem ara seran:

- **NextStep.** Aquesta acció canvia el taulell a la següent generació i es produeix amb l'esdeveniment de tecla `N` polsada.
- **SetCell (x, y).** Aquesta acció canvia la casella *clickada* d'estat viva a morta o viceversa i es produeix amb l'esdeveniment de botó del ratolí polsat.

Per obtenir la posició de la casella (columna, fila) a partir de les coordenades del ratolí (x, y), considerarem per ara que cada casella ocupa 1x1 unitats del sistema de coordenades, i que l'origen de coordenades (centre de la vista) correspon a la posició (0, 0) del taulell.

Així doncs, per ara, la conversió de coordenades gràfiques a posicions en el taulell consistirà senzillament en usar la funció `round` que obté, a partir d'un valor de tipus `Double`, el seu arrodoniment (de tipus `Int`):

```
round :: (RealFrac a, Integral b) => a -> b
```

La funció `view` és l'encarregada de configurar les funcions que obtenen les corresponents accions a partir dels esdeveniments de teclat o ratolí, i d'obtenir el dibuix.

Per a la visualització de l'estat usarem la funció `drawBoard` del mòdul `Life.Draw` (fitxer [life/Life/Draw.hs](#)) que haureu de completar.

Per això, definiu una funció

```
drawCell :: Pos -> Drawing
```

que dibuixi un rectangle sòlid de dimensions 1x1 en la coordenada corresponent a la posició indicada. Useu la funció `fromIntegral` per a la conversió de `Int` a `Double`.

Un cop definida `drawCell`, la funció `drawBoard` la podeu definir fàcilment usant les funcions `liveCells`, `foldMap` i `drawCell`.

Treball

1. Completeu el mòdul `Life.Draw` (fitxer `Life/Draw.hs`).
2. Executeu el programa `life` fent:

```
stack build
stack run life
```
3. Visiteu la URL `http://localhost:3708/` i comproveu que funciona correctament.

1.3 Segon pas

El segon pas consistirà en afegir un control per mostrar/ocultar una graella en la visualització del taulell.

Per això, definim el tipus `GridMode` per representar el mode de graella i un nou camp `gmGridMode` en l'estat del joc `Game`:

```
data Game = Game
  { gmBoard :: Board      — last board generation
  , gmGridMode :: GridMode
  }

data GridMode = NoGrid | LivesGrid | ViewGrid
```

Els valors possibles de `GridMode` són:

- **NoGrid**: Visualitza sense graella.
- **LivesGrid**: Visualitza amb una graella que inclou les cel·les vives.
- **ViewGrid**: Visualitza amb una graella que ocupa tota la vista.

El mode de graella de l'estat inicial serà **NoGrid**:

```
initial = Game
  { gmBoard = foldr (setCell True) initBoard board0Cells
  , gmGridMode = NoGrid
  }
```

Haureu d'afegir l'acció **ChangeGridMode** i el tractament de l'esdeveniment de tecla **G** polsada que obtingui aquesta acció. L'acció haurà de canviar el mode de graella entre els 3 valors possibles:

```

data Action =
    ...
    | ChangeGridMode
...
update ChangeGridMode = do
    game <- get
    let newMode =
        — obtenir el següent mode a partir
        — del mode actual 'gmGridMode game'
        ...
    put game{ gmGridMode = newMode }
...
keyMap "G" = ChangeGridMode
...

```

També haureu d'extendre la funció de visualització per a que generi el dibuix de la graella segons el mode de l'estat actual. En mode **LivesGrid**, els extrems de la graella els podeu obtenir amb les funcions **minLiveCell** i **maxLiveCell** del mòdul **Life.Board**.

En mode **ViewGrid**, els extrems de la graella els haureu d'obtenir a partir de les coordenades dels extrems de la vista **viewWidth** i **viewHeight**.

Treball

1. Completeu el mòdul **Main** (fitxer **life.hs**).
2. Executeu el programa:


```
stack run life
```
3. Visiteu la URL <http://localhost:3708/> i comproveu que funciona correctament.

1.4 Tercer pas

En el tercer pas introduïrem controls per poder fer *zoom* i desplaçaments en la visualització del taulell.

Per això, definim nous camps **gmZoom** i **gmShift** en l'estat del joc **Game**:

```

data Game = Game
    { gmBoard :: Board          — last board generation
    , gmGridMode :: GridMode
    , gmZoom :: Double, gmShift :: Point
    }

```

Inicialment no tindrem *zoom* ni desplaçament:

```

initial = Game
    { gmBoard = foldr (setCell True) initBoard board0Cells
    , gmGridMode = NoGrid
    , gmZoom = 1.0, gmShift = (0.0, 0.0)
    }

```

```
}
```

Haureu d'afegir noves accions per canviar l'estat de `gmZoom` i `gmShift`:

```
data Action =  
    ...  
    | ZoomOut  
    | ZoomIn  
    | AddShift (Double, Double)
```

Tracteu els esdeveniments de tecles 0/I pulsades per l'accions `ZoomOut/ZoomIn` (reduir/ampliar).

Pot ser convenient limitar el rang dels possibles valors del camp `gmZoom`. Per exemple:

```
...  
update ZoomIn = do  
    game <- get  
    when (gmZoom game < 2.0) $  
        put game{ gmZoom = gmZoom game * 2.0 }  
...
```

Tracteu els esdeveniments de tecles de direcció (fletxes) pulsades per l'acció `AddShift dir` (afegir desplaçament de la vista del taulell):

```
...  
keyMap "ARROWUP"      = Just $ AddShift ( 0, -1) -- Down shift  
keyMap "ARROWDOWN"    = Just $ AddShift ( 0,  1) -- Up shift  
keyMap "ARROWRIGHT"   = Just $ AddShift (-1,  0) -- Left shift  
keyMap "ARROWLEFT"    = Just $ AddShift ( 1,  0) -- Right shift  
...
```

En el tractament de l'acció `AddShift` dins la funció `update`, us serà molt convenient usar les funcions del mòdul `Drawing.Vector` de la biblioteca.

En el tractament de l'acció `SetCell`, per obtenir la posició de la casella (columna, fila) a partir de les coordenades del ratolí (x, y), us serà molt útil definir i usar la següent funció:

```
pointToPos :: Game -> Point -> Pos  
pointToPos game p =  
    let (gx, gy) = (1.0 / gmZoom game) *^ p ^- gmShift game  
    in (round gx, round gy)
```

Aquesta funció també us pot ajudar, en la visualització de l'estat, per obtenir les posicions corresponents als extrems de la vista.

També haureu de modificar la funció de visualització, aplicant l'escalat i traslació adequats segons els valors actuals de `gmZoom` i `gmShift`.

Treball

1. Completeu el mòdul `Main` (fitxer `life.hs`).
2. Executeu el programa:

```
stack run life
```
3. Visiteu la URL `http://localhost:3708/` i comproveu que funciona correctament.

1.5 Quart pas

En aquest pas afegirem en el dibuix text que mostri l'estat d'alguns paràmetres del joc i una ajuda de les tecles que canvien els paràmetres del joc (veieu el [funcionament de la versió final](#)).

Per això, definim el camp `gmShowHelp` en l'estat del joc `Game`:

```
data Game = Game
  { gmBoard  :: Board      — last board generation
  , gmGridMode :: GridMode
  , gmZoom  :: Double, gmShift :: Point
  , gmShowHelp :: Bool
  }
```

Inicialment mostrarem l'ajuda:

```
initial = Game
  { gmBoard = foldr (setCell True) initBoard board0Cells
  , gmGridMode = NoGrid
  , gmZoom = 1.0, gmShift = (0.0, 0.0)
  , gmShowHelp = True
  }
```

Haureu d'afegir una nova acció per canviar l'estat de mostrar ajuda:

```
data Action =
  ...
  | ToggleHelp
```

Tracteu l'esdeveniment de tecla `H` pulsada per l'acció `ToggleHelp`:

```
...
update ToggleHelp =
  modify $
    \game -> game{ gmShowHelp = not $ gmShowHelp game }
...
keyMap "H" = Just ToggleHelp
```

...

I també haureu de modificar la funció de dibuix, mostrant l'estat i l'ajuda segons el valor actual de `gmShowHelp`.

Treball

1. Modifiqueu la funció de dibuix del mòdul `Main` de la versió anterior, per tal que mostri l'estat i una ajuda del joc.
2. Executeu el programa:

```
stack run life
```
3. Visiteu la URL i comproveu que funciona correctament.

1.6 Cinquè pas

En aquest últim pas afegirem la funcionalitat de guardar i llegir en format JSON l'estat del taulell en un fitxer amb un nom determinat fixe (per exemple "board.json").

Per això, haureu d'afegir noves accions per iniciar l'operació de guardar, iniciar l'operació de llegir i notificar el final de la lectura:

```
data Action =  
    ...  
    | StartLoad  
    | EndLoad Board  
    | Save
```

Modifiqueu adequadament la funció `update` amb el corresponent tractament d'aquestes accions. Per això haureu d'usar, entre altres, les funcions `deferSub` i `deferIO_` del paquet `drawing-activity`, i les funcions `encode` i `eitherDecode` del paquet `aeson`.

Observeu que les instàncies per a la conversió del taulell (de tipus `Board`) a i des de JSON, usades per les funcions `encode` i `eitherDecode`, ja se us donen fetes (veieu el mòdul `Life.Board`).

Tracteu l'esdeveniment de tecla `L` polsada per l'acció `StartLoad` i l'esdeveniment de tecla `S` polsada per l'acció `Save`:

```
...  
keyMap "L" = Just StartLoad  
keyMap "S" = Just Save  
...
```

Treball

1. Modifiqueu la funció **update** del mòdul **Main** de la versió anterior.
2. Executeu el programa:
`stack run life`
3. Visiteu la URL i comproveu que funciona correctament.

2 Codi font del projecte

2.1 Configuració del projecte *stack*

```
# Descomenteu un dels 'resolver's':

# resolver: lts-18.28
# (using ghc-8.10.7)

# resolver: lts-20.26
# (using ghc-9.2.8)

packages:
- life
- drawing-activity
- drawing-core
- jsaddle-warp-extra

extra-deps:
- jsaddle-0.9.9.3@sha256:
    b2cd9ef8f97d4a1e28d4ee6ecf3c9681a2150a82ec5205811dc7d0334d20be5
    ,4708
- miso-1.8.7.0@sha256:7481
    cffa5318c0e69c76a4c7b0d541670ba1e1aa509e5d297dcafcc96376ac56
    ,3934
# servant-lucid-0.9.0.6 is necessary for resolver <= lts-18.28
- servant-lucid-0.9.0.6@sha256:
    b7de9f07a056037a10cc012136e7cab2e685403791d2c0cb0a92f16e3ade6ac
    ,1877
- jsaddle-warp-0.9.9.5@sha256:
    ab68bd3720fb97078c5ae7545a7ebee05ac01f5c169f05365eb2cd28fcce1a4a
    ,4910
```

Listing 1: stack.yaml

2.2 Paquet life

2.2.1 Descripció del paquet

```
cabal-version: 1.12

name:          life
version:       0.0.0.0
author:        Nom de l'estudiant
build-type:    Simple

executable life
  main-is:
    life.hs
  other-modules:
    Life.Board
    Life.Draw
```

```

build-depends:
  base >=4.7 && <5
  , containers
  , drawing-activity >=0.2
  , drawing-core >=0.1
  , text
default-language: Haskell2010

```

Listing 2: life/life.cabal

2.2.2 Mòdul Main

```

{-# LANGUAGE OverloadedStrings #-}

module Main where
import Life.Board
import Life.Draw

import Drawing
import Drawing.Activity

import qualified Data.Text as T

-- The game state

data Game = Game
  { gmBoard :: Board      -- last board generation
  }

-- Initialization

viewWidth, viewHeight :: Double
viewWidth = 60.0
viewHeight = 30.0

main =
  activityOf 3708 viewWidth viewHeight initial update view

board0Cells =
  [(-5, 0), (-4, 0), (-3, 0), (-2, 0), (-1, 0), (0, 0), (1, 0),
   (2, 0), (3, 0), (4, 0)]

initial = Game
  { gmBoard = foldr (setCell True) initBoard board0Cells
  }

-- Update state

data Action =
  NextStep
  | SetCell Point
  deriving (Eq, Show)

```

```

update :: Action -> State Game ()
update NextStep =                — Next generation
    modify $
        \game -> game{ gmBoard = nextGeneration (gmBoard game) }
update (SetCell point) = do      — Set live/dead cells
    game <- get
    let pos = pointToPos point
        brd = gmBoard game
    put game{ gmBoard = setCell (not $ cellIsLive pos brd) pos brd
    }

pointToPos :: Point -> Pos
pointToPos (x, y) = (round x, round y)

— View state

view :: Game -> View Action
view game =
    drawingView
        [ onKeyDown' keyMap
          , onMouseDown SetCell
          ]
    $ draw game

keyMap :: T.Text -> Maybe Action
keyMap "N" = Just NextStep
keyMap _   = Nothing

draw game =
    drawBoard (gmBoard game)

```

Listing 3: life/life.hs

2.2.3 Mòdul Life.Board

```

— This module defines a model for the board of 'The Game of Life'

module Life.Board
    ( Board, Pos, initBoard
    , cellIsLive, liveCells
    , minCol, maxCol, minRow, maxRow, minLiveCell, maxLiveCell
    , setCell, nextGeneration
    )
where
import qualified Data.Set as S
import qualified Data.Map as M

import System.IO.Unsafe

— Each cell in the board is determined by a pair of Int's (column,
row)
type Pos = (Int, Int)

```

```

— The type of the board
data Board = Board
    { cells :: S.Set Pos
      , minCol :: Int, maxCol :: Int
      , minRow :: Int, maxRow :: Int
    }
    deriving (Eq, Show, Read)

— The empty board (all cells are dead)
initBoard :: Board
initBoard =
    Board S.empty 0 0 0 0

— 'cellsLive pos board' is true iff the cell at 'pos' of 'board'
  is live
cellsLive :: Pos -> Board -> Bool
cellsLive pos board =
    S.member pos (cells board)

— 'liveCells board' is the list of live cells of 'board'
liveCells :: Board -> [Pos]
liveCells board =
    S.toList (cells board)

— Get the minimum position (column, row) of the live cells
minLiveCell :: Board -> Pos
minLiveCell board =
    (minCol board, minRow board)

— Get the maximum position (column, row) of the live cells
maxLiveCell :: Board -> Pos
maxLiveCell board =
    (maxCol board, maxRow board)

— 'setCell live pos board' change the liveness of cell at 'pos' of
  'board' (live if 'live' is 'True' or dead if 'live' is 'False'
  ')
setCell :: Bool -> Pos -> Board -> Board
setCell False pos board =
    board{ cells = S.delete pos (cells board) }
setCell True pos@(col, row) board | S.null (cells board) =
    board{ cells = S.insert pos (cells board)
          , minCol = col, maxCol = col
          , minRow = row, maxRow = row
          }
setCell True pos@(col, row) board =
    board{ cells = S.insert pos (cells board)
          , minCol = min col (minCol board), maxCol = max col (
            maxCol board)
          , minRow = min row (minRow board), maxRow = max row (
            maxRow board)
          }

— 'nextGeneration board' gets the next generation of 'board'
nextGeneration :: Board -> Board
nextGeneration board =
    foldr setLive initBoard $ M.toList newCells

```

```

where
  — 'newCells' is a map from cell positions to pairs (live,
  neighbors)
  newCells :: M.Map Pos (Bool, Int)
  newCells =
    let livePoss = liveCells board
        map0 = M.fromList $ fmap (\pos -> (pos, (True, 0)))
    livePoss
    in foldr newCellsFor map0 livePoss
  newCellsFor pos1 map1 =
    let nposs = neighborPositions pos1
        incNeighbors pos map = M.insertWith (\ - (live, n)
-> (live, n + 1)) pos (False, 1) map
    in foldr incNeighbors map1 nposs
  setLive (pos, (live, around)) board1 =
    if live && elem around [2, 3] || not live && around ==
3 then
    setCell True pos board1
  else board1

neighborPositions :: Pos -> [Pos]
neighborPositions (col, row) =
  [ (col + 1, row), (col + 1, row + 1), (col, row + 1), (col - 1,
    row + 1)
  , (col - 1, row), (col - 1, row - 1), (col, row - 1), (col + 1,
    row - 1)
  ]

— JSON representation

instance FromJSON Board where
  parseJSON = withObject "Board" $ \o -> Board
    <$> o .: "cells"
    <*> o .: "minCol" <*> o .: "maxCol"
    <*> o .: "minRow" <*> o .: "maxRow"

instance ToJSON Board where
  toJSON (Board cells minCol maxCol minRow maxRow) =
    object [ "cells" .= cells
            , "minCol" .= minCol, "maxCol" .= maxCol
            , "minRow" .= minRow, "maxRow" .= maxRow
            ]

```

Listing 4: life/Life/Board.hs

2.2.4 Mòdul Life.Draw

```

module Life.Draw where
import Life.Board
import Drawing

drawBoard :: Board -> Drawing
drawBoard board =
  — A completar per l'estudiant

```



```

...
— 'drawGrid minPos maxPos' obte el dibuix d'una graella que inclou
  els 2 extrems indicats.
— 'minPos' es la posicio de l'extrem (esquerra, inferior) i
— 'maxPos' es la posicio de l'extrem (dreta, superior)'.
— NOTA: Aquesta funcio s'usa a partir del segon pas de la practica
.
drawGrid :: Pos -> Pos -> Drawing
drawGrid minPos maxPos =
  colored gray $
    foldMap vline [fst minPos .. fst maxPos + 1]
    <> foldMap hline [snd minPos .. snd maxPos + 1]
  where
    hline row = polyline [ posToCoords (fst minPos, row),
      posToCoords (fst maxPos + 1, row) ]
    vline col = polyline [ posToCoords (col, snd minPos),
      posToCoords (col, snd maxPos + 1) ]
    posToCoords (col, row) = (fromIntegral col - 0.5,
      fromIntegral row - 0.5)

```

Listing 5: life/Life/Draw.hs